kernel.h: A Cross-Vendor GPU Kernel Compiler from a C++23 DSL through Structured SSA and MLIR to NVVM/ROCDL

kernel.h project

October 2026

Abstract

`kernel.h` is a GPU kernel compiler. It takes one kernel description in a C++23 embedded DSL, lowers it through a custom structured-SSA IR called atom, runs a seven-pass pipeline, lowers to an out-of-tree MLIR dialect, and emits native code for two vendors: PTX and cubin for NVIDIA sm_80, and AMDGCN ISA and HSACO for AMD gfx1101. The compiler uses MLIR (LLVM 21.1.8) for the atom dialect, the `gpu`/`scf`/`vector` conversion, and the vendor serializers. The DSL, atom IR, and the seven passes use only the project's C++ code. A thin HIP runtime loads the code objects with `hipModuleLoadData`, launches kernels at the device ABI, and packs memref descriptors into launch arguments. A second backend binds the CUDA Driver API at runtime with `dlopen`. The report covers the IR design (D10), the pass pipeline with before and after dumps, the device-only lowering, the vendor serializers, the autotuner and a correctness bug its numeric gate caught, and measurements on an AMD Radeon RX 7800 XT (gfx1101): 347.8 GB/s DRAM, 15 559.8 GFLOPS FMA, 1072.5 GFLOPS GEMM at 8192^3 (about 9.5× below rocBLAS), 1110 GB/s fused GELU, and 43.6 GFLOPS attention. It also records the limits: tensor cores are out of scope, shared memory is static, the GEMM has no double buffering, and the NVIDIA path is cross-compiled locally and was run on an RTX 6000 Ada pod.

Contents

1 Introduction and System Architecture **3**
- 1.1 Goals and Scope . . . 3
- 1.2 Architecture Overview . . . 3

2 IR Design: Structured SSA **3**
- 2.1 Rationale (decision D10) . . . 3
- 2.2 Op Inventory . . . 4
- 2.3 A Verbatim `.atom` Excerpt . . . 4

3 The Pass Pipeline **5**
- 3.1 Before/After Excerpts . . . 5

4 MLIR Lowering **6**
- 4.1 Device-Only Decision and Rationale . . . 7

5 Vendor Backends **8**
- 5.1 Verbatim PTX (NVIDIA, cross-compiled sm_80) . . . 8
- 5.2 Verbatim gfx1101 ISA (AMD, executed locally) . . . 9
- 5.3 Serialization Mechanics . . . 9

6 Runtime and Loading **10**
- 6.1 Descriptor ABI . . . 10
- 6.2 CUDA Backend (cross-ready, not linked) . . . 10

7 Autotuning **11**

8 Performance **11**
- 8.1 Measured Roofline Ceilings (this box) . . . 11
- 8.2 GEMM: kernelh vs. rocBLAS . . . 12
- 8.3 Elementwise / Reduction Kernels . . . 13
- 8.4 Occupancy . . . 14

9 Validation and Failure Evidence **14**

10 Limitations **15**

11 Future Work **15**

12 Conclusion **16**

1 Introduction and System Architecture

1.1 Goals and Scope

`kernel.h` compiles one kernel description into NVIDIA and AMD GPU machine code, runs it on the local AMD card, checks the output against an fp64 CPU reference, and benchmarks it against AMD's vendor library (rocBLAS). The compiler uses MLIR ([1], LLVM 21.1.8) for the atom dialect, the `gpu/scf/vector` conversion, and the vendor serializers. The DSL, atom IR, and the seven passes are C++ code with no MLIR dependency (§2).

Two compile entry points exist, both run in-process:

- `kernelh-opt`: the debug and test driver. It parses `.atom` text, runs passes, prints IR, and provides an `mlir-opt`-style tool with the atom dialect registered.
- `kernelh`: the production CLI. Its subcommands are `compile`, `explore`, `bench`, and `tune`. It links the DSL, IR, passes, emitter, backends, and runtime in-process.

1.2 Architecture Overview

The toolchain is shown in fig. 1. A kernel description flows from the C++23 DSL builder through atom, the seven-pass pipeline, the atom MLIR dialect, and the `gpu/scf/vector` device module to the NVVM/ROCDL serializers. The runtime loads the resulting HSACO or cubin through the HIP module loader or the `dlopen`ed CUDA Driver API backend and launches it on the GPU. End-to-end tests compare the output against an fp64 reference, and the benchmark harness compares it against rocBLAS.

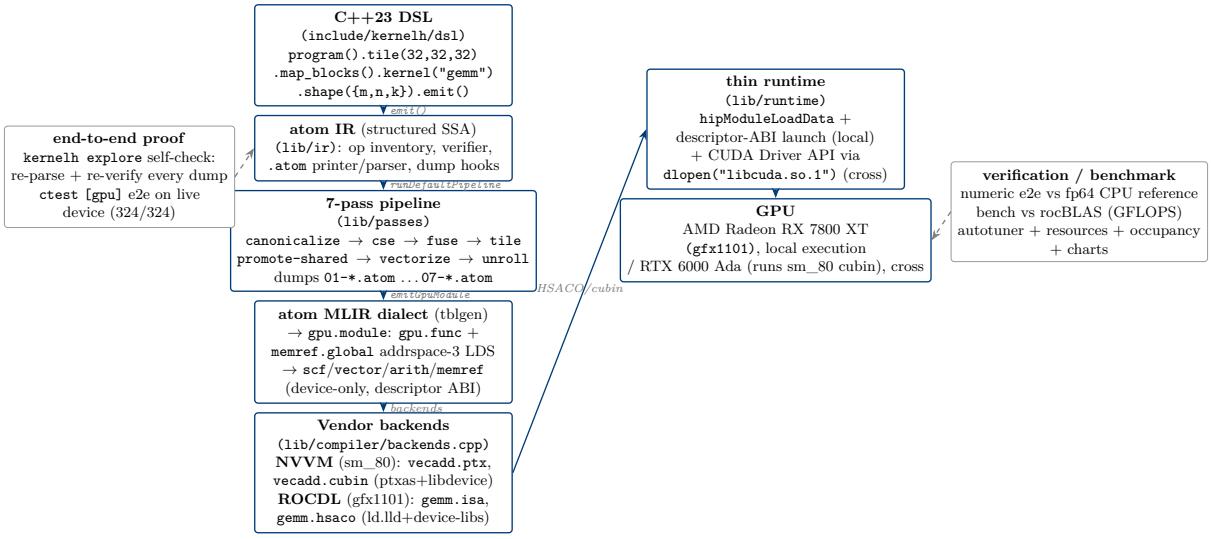


Figure 1: End-to-end architecture of `kernel.h`: C++23 DSL to atom IR, seven passes, the atom MLIR dialect, `gpu/scf/vector`, NVVM/ROCDL, then PTX/cubin/HSACO/ISA. The thin HIP/CUDA runtime loads the code object and launches it on the GPU.

The host measurement platform is the AMD Radeon RX 7800 XT (gfx1101, Navi 32): 30 compute units, 16 GiB VRAM (17 163 091 968 bytes), 64 KiB LDS per workgroup, and wavefront size 32. It runs in full runtime mode (`degraded=false`, §9).

2 IR Design: Structured SSA

2.1 Rationale (decision D10)

The atom IR is *structured SSA*: nested block scopes with values dominating within their scope. Control flow uses `atom.loop_for` with `iter_args() -> (...)`. The loop is a structured loop and the only SSA merge point; loop-carried values are its iter_args, terminated by `atom.yield`. `atom.reduce` (sum or max over a tensor dim) is built on `loop_for` and iter_args.

Three properties drive the design:

1. **Reductions stay in the IR.** Softmax's running max and sum, GEMM's accumulator, scan's carry, and attention's online statistics are loop-carried iter_args. They need no conversion to a generic CFG. The pass pipeline, fusion in particular, inspects loop domains and barriers. A structured form makes "same iteration domain, same barrier state" a local check.
2. **The verifier is structural.** Dominance, barrier placement, operand types, and shared-memory totals are structural properties. `verifyModule` reports every violation in one pass; it collects all diagnostics and does not stop at the first. The round-trip self-check in `kernelh explore` uses this (§3).
3. **The text round-trips.** `.atom` is both printer output and parser input. A deterministic value-numbering scheme (an MLIR-style active-name set, where nested scopes do not shadow a live outer name) makes every per-pass dump re-parseable and verifiable.

2.2 Op Inventory

Types: `f16`, `f32`, `i32`, `i64`, `index` scalars; `tensor<...xE, space>` with `space` ∈ {`global`, `shared`}; `vector<WxE>`. Ops are plain data (`enum class OpKind` in `include/kernelh/ir/ops.hpp`). table 1 lists the pinned inventory.

Table 1: atom IR op inventory (pinned).

kind	purpose
`atom.const`	scalar constant (int or float payload)
`atom.add/sub/mul/div/fma/max/min`	scalar/vector arithmetic
`atom.cmp` + `atom.select`	comparison (result `i32` carrier) + select
`atom.math{exp,log,sqrt,rsqrt,tanh}`	math intrinsics
`atom.load` / `atom.store`	tensor load/store with affine index expressions
`atom.load_vec` / `atom.store_vec`	width-attribute vector memory ops
`atom.alloc_shared`	static shared memory, size from type
`atom.barrier`	workgroup barrier (top level of loop bodies only)
`atom.thread_id` / `block_id` / `block_dim` / `grid_dim` {dim}	launch intrinsics
`atom.loop_for`	structured loop with iter_args + `atom.yield`
`atom.reduce`	sum/max over a tensor dim (built on loop_for)
`atom.copy`	tile copy helper (global→shared staging)
`atom.kernel`	container with schedule attrs (`tile`, `map_blocks`, `map_warps`, `use_shared_memory`, `vector_width`, `unroll`, `shape`)

2.3 A Verbatim `.atom` Excerpt

listing 1 is the verbatim pre-pipeline input of the GEMM explore run (`artifacts/run-report-amd/00-input.atom`, emitted by `kernelh explore -kernel gemm -shape m=256,n=256,k=256 -target amdgcn-gfx1101`). It is the tiled shared-memory GEMM: a flat 2048-`f32` LDS buffer holds per-wavefront A row-slices and the B tile; the k-tile loop carries the `f32` accumulator as an iter_arg. (Schedule attrs are printed in a fixed order; `shape` is key-sorted; float constants round-trip shortest.)

Listing 1: Pre-pipeline GEMM input (`00-input.atom`).

```
atom.kernel @gemm(%v0: tensor<256x256xf32, global>, %v1: tensor<256x256xf32, global>, %v2: tensor<256x256xf32, global>)
      {
  tile = [32, 32, 32]
  map_blocks = true
  map_warps = true
  use_shared_memory = true
  vector_width = 1
  unroll = 1
  shape = {k = 256, m = 256, n = 256}
  %v3 = atom.const 0 : index
  ...
  %v15 = atom.thread_id x : index
  %v16 = atom.block_dim x : index
  %v17 = atom.div %v15, %v12 : index
  %v18 = atom.mul %v17, %v9 : index
  %v19 = atom.mul %v17, %v12 : index
  %v20 = atom.sub %v15, %v19 : index
  %v21 = atom.alloc_shared : tensor<2048xf32, shared>
```

```
  atom.loop_for %v3 to %v4 step %v7 iter_args() -> () {
    ^bb0(%v22: index):
      ...
      %v28 = atom.loop_for %v3 to %v6 step %v9 iter_args(%v14) -> (f32) {
        ^bb0(%v29: index, %v30: f32):
          ...
          atom.barrier
          %v33 = atom.loop_for %v3 to %v32 step %v7 iter_args(%v30) -> (f32) {
            ^bb0(%v34: index, %v35: f32):
              %v36 = atom.add %v18, %v34 : index
              %v37 = atom.load %v21[%v36] : tensor<2048xf32, shared>
              ...
              %v42 = atom.fma %v37, %v41, %v35 : f32
              atom.yield %v42 : f32
          }
          atom.barrier
          atom.yield %v33 : f32
      }
      atom.store %v28, %v2[%v22, %v27] : tensor<256x256xf32, global>
      atom.yield
  }
}
```

3 The Pass Pipeline

Canonical order, pinned by `kernelh::passes::defaultPipelineOrder()` in `lib/passes/pipeline.cpp`. Pipeline-order lock tests assert the sequence:

`canonicalize` → `cse` → `fuse` → `tile` → `promote-shared` → `vectorize` → `unroll`

Every pass is a self-registering `.cpp` in `lib/passes/` (GLOB + a file-scope static initializer; consumers force-load the archive with `$<LINK_LIBRARY:WHOLE_ARCHIVE,kernelh_passes>`). `kernelh explore` runs this exact pipeline and dumps `00-input.atom` + `01-<pass>.atom` … `07-unroll.atom`, then re-parses and re-verifies every dump in-process, so a corrupted dump stops the run. fig. 2 shows the pipeline; §3.1 gives before and after excerpts from `artifacts/run-report-amd/` and `artifacts/run-report-fused/`.

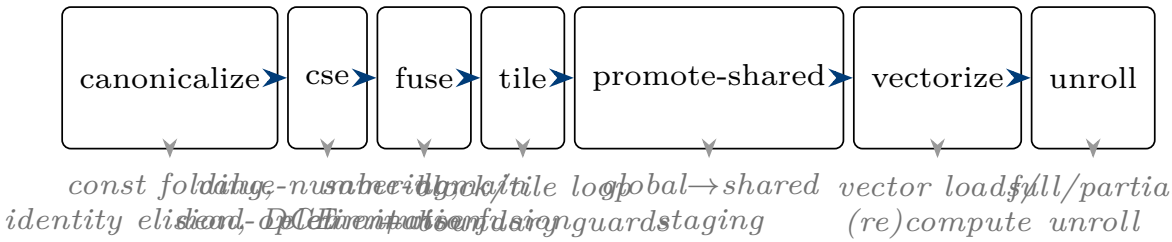


Figure 2: The seven-pass pipeline, in canonical order (`kernelh::passes::defaultPipelineOrder()`).

3.1 Before/After Excerpts

canonicalize (01). GEMM's pre-pipeline input has no foldable constants: all `atom.const` values are leaves. `01-canonicalize.atom` is byte-identical to `00-input.atom`. The fusion example below shows a case where canonicalize did rewrite: the `x + 0.0` identity elision is what lets the vectorizer widen the fused chain.

cse (02). `02-cse.atom` (3258 B) is smaller than `00-input.atom` (3461 B). CSE merged duplicate constants (`%v8 = atom.const 32` and `%v9 = atom.const 32`, which are different SSA ids with the same value, plus the duplicate `1024` constants) and rewrote every use to the survivor.

tile (04). The domain loop `atom.loop_for %v3 to %v4 step %v7` (0..256 step 1) is rewritten into a block/tile loop. After the pass the kernel body starts with `atom.block_id x`, and the domain loop becomes:

Listing 2: Tiled domain loop (`04-tile.atom`).

```
  %v16 = atom.block_id x : index
  %v17 = atom.const 32 : index
```

```
%v18 = atom.mul %v16, %v17 : index          ; base = block_id * 32
...
atom.loop_for %v3 to %v28 step %v5 iter_args() -> () {
  ^bb0(%v29: index):
    %v30 = atom.mul %v29, %v25 : index
    %v31 = atom.add %v30, %v21 : index      ; row = i*W + wf
    %v32 = atom.add %v18, %v31 : index      ; gi = base + row
    %v33 = atom.cmp lt %v31, %v17 : index   ; ok_tile guard
    ...
```

The tile guard (`select(ok, gi, 0)`) wraps exactly the A-load and the C-store, the only two uses of the induction `i`. GEMM's `04` is identical to `05/06/07`: promote-shared is a no-op on the pre-staged template, and `vector_width=1` with `unroll=1` leaves the IR unchanged. The explore self-check asserts this idempotency.

Fusion before and after. The fused_gelu explore run (`kernelh explore -kernel fused_gelu -shape n=1048576 -target amdgcn-gfx1101`) produced the two dumps below. Before (`02-cse.atom`): two same-domain loops with an intermediate `tensor<1048576xf32>` staging buffer. After (`03-fuse.atom`): one loop computes the whole chain pointwise. The intermediate buffer is gone, along with the producer store and consumer load.

Listing 3: Fusion before (`02-cse.atom`): two loops + staging buffer.

```
atom.loop_for %v15 to %v3 step %v16 iter_args() -> () {
  ^bb0(%v17: index):
    %v18 = atom.load %v0[%v17] : tensor<1048576xf32, global>
    %v19 = atom.fma %v4, %v18, %v5 : f32
    atom.store %v19, %v1[%v17] : tensor<1048576xf32, global>
    atom.yield
}
atom.loop_for %v14 to %v3 step %v15 iter_args() -> () {
  ^bb0(%v16: index):
    %v17 = atom.load %v1[%v16] : tensor<1048576xf32, global>
    %v18 = atom.mul %v17, %v17 : f32
    %v19 = atom.mul %v18, %v17 : f32
    ... tanh ...
    atom.store %v27, %v2[%v16] : tensor<1048576xf32, global>
    atom.yield
}
```

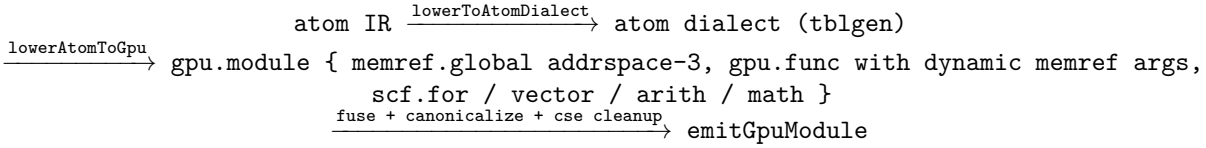
Listing 4: Fusion after (`03-fuse.atom`): one loop, buffer gone.

```
atom.loop_for %v14 to %v3 step %v15 iter_args() -> () {
  ^bb0(%v16: index):
    %v17 = atom.load %v0[%v16] : tensor<1048576xf32, global>
    %v18 = atom.fma %v4, %v17, %v5 : f32
    %v19 = atom.mul %v18, %v18 : f32
    %v20 = atom.mul %v19, %v18 : f32
    %v21 = atom.mul %v8, %v20 : f32
    %v22 = atom.add %v18, %v21 : f32
    %v23 = atom.mul %v7, %v22 : f32
    %v24 = atom.math tanh %v23 : f32
    %v25 = atom.add %v24, %v6 : f32
    %v26 = atom.mul %v5, %v18 : f32
    %v27 = atom.mul %v26, %v25 : f32
    atom.store %v27, %v2[%v16] : tensor<1048576xf32, global>
    atom.yield
}
```

Fusion fires only on same-domain producer and consumer pairs with no barrier between them. The barrier-separated negative fixture (`test/fixtures/chain_barrier.atom`) is left unchanged. This applies the elementwise fusion used by FlashAttention-style kernels [2, 3] at the IR level. The tile pass follows the polyhedral tiling approach used by Tiramisu [4], expressed in a small structured-SSA form.

4 MLIR Lowering

The lowering chain, all in `lib/compiler/emitter.cpp` (two stages), is:



4.1 Device-Only Decision and Rationale

The emitter produces a `gpu.module` with `gpu.func` kernels. It does not emit `gpu.launch`, outline host code, or use `gpu-to-llvm` host machinery. kernel.h's runtime (§6) launches the kernels at the device ABI, so host launching stays in the runtime and the compiler does not depend on the MLIR ExecutionEngine. The device module is what the vendor serializers consume (`gpu-module-to-binary`), and the descriptor ABI (§6.1) supplies the launch arguments. The runtime links no MLIR.

listing 5 is the MLIR output, verbatim from `artifacts/run-report-nv/gpu.mlir`. The vecadd `gpu.module` has the same shape on the AMD side.

Listing 5: Verbatim `gpu.module` for vecadd (`gpu.mlir`).

```
module {
  gpu.module @kernelh {
    gpu.func @vecadd(%arg0: memref<?xf32>, %arg1: memref<?xf32>, %arg2: memref<?xf32>) attributes {
     kernel} {
      %c1024 = arith.constant 1024 : index
      %block_id_x = gpu.block_id  x
      %block_dim_x = gpu.block_dim  x
      %grid_dim_x = gpu.grid_dim  x
      %thread_id_x = gpu.thread_id  x
      %0 = arith.muli %block_id_x, %block_dim_x : index
      %1 = arith.addi %0, %thread_id_x : index
      %2 = arith.muli %block_dim_x, %grid_dim_x : index
      scf.for %arg3 = %1 to %c1024 step %2 {
        %3 = memref.load %arg0[%arg3] : memref<?xf32>
        %4 = memref.load %arg1[%arg3] : memref<?xf32>
        %5 = arith.addf %3, %4 : f32
        memref.store %5, %arg2[%arg3] : memref<?xf32>
      }
      gpu.return
    }
  }
}
```

Static shared memory uses addrspace-3 globals. A per-thread `alloca` would break cross-thread exchange, because NVPTX treats `alloca addrspace(3)` as per-thread. The GEMM shared buffer from `artifacts/run-report-amd/rocdl.mlir`:

Listing 6: Static-LDS lowering in the ROCDL module (`rocdl.mlir`).

```
  gpu.module @kernelh [#rocdl.target<chip = "gfx1101">] attributes {llvm.data_layout = ...} {
    llvm.mlir.global external @gemm_shared_0() {addr_space = 3 : i32} : !llvm.array<2048 x f32> {
      %0 = llvm.mlir.undef : !llvm.array<2048 x f32>
      llvm.return %0 : !llvm.array<2048 x f32>
    }
    llvm.func @gemm(%arg0: !llvm.ptr, %arg1: !llvm.ptr, %arg2: i64, ... ) attributes {gpu.kernel, rocdl.
     kernel} {
      ...
```

Key lowering facts:

- Kernel arguments are **dynamic-shape memref descriptors** (`memref<?xf32>`, `memref<?x?xf32>`). The descriptor ABI expands a rank-N buffer to $2N + 3$ argument slots: allocated ptr, aligned ptr, offset i64, N sizes, N strides (§6.1).
- Loop bounds for dynamic args come from `memref.dim` at runtime, not the static atom type.
- `atom.reduce` lowers to nested `scf.for` + iter_args; `atom.copy` to nested `scf.for` + memref loads/stores.

- Custom cmp (i32 carrier) lowers to `arith.extui`, and select lowers to `arith.trunci`. Canonicalize folds `trunci(extui(cmp))` back to a plain `cmpf` plus `select`.
- The emitter's literal `kernel` marker attr is normalized to `gpu.kernel` inside the backend shim (`normalizeKernelMarkers`) so NVPTX/ROCDL emit `.visible .entry` / proper kernels.

5 Vendor Backends

Two vendor paths in `lib/compiler/backends.cpp`; each consumes a *fresh clone* of the `gpu.module` per artifact because `gpu-module-to-binary` mutates the module in place. Both paths use LLVM's code-generation infrastructure [5], invoked through the MLIR `gpu`-to-NVVM/ROCDL conversions (§4).

5.1 Verbatim PTX (NVIDIA, cross-compiled sm_80)

listing 7 is verbatim from `artifacts/run-report-nv/vecadd.ptx` (the whole kernel of the explore run `kernelh explore -kernel vecadd -shape n=1024 -target nvptx-sm_80`):

Listing 7: PTX for vecadd, sm_80 (`vecadd.ptx`).

```
// Generated by LLVM NVPTX Back-End
//
.version 7.0
.target sm_80
.address_size 64

.visible .entry vecadd(
  .param .u64 .ptr .align 1 vecadd_param_0,
  .param .u64 .ptr .align 1 vecadd_param_1,
  .param .u64 vecadd_param_2,
  .param .u64 vecadd_param_3,
  .param .u64 vecadd_param_4,
  .param .u64 .ptr .align 1 vecadd_param_5,
  ...
  .param .u64 vecadd_param_14
)
{
  .reg .pred  %p<3>;
  .reg .b32   %r<8>;
  .reg .b64   %rd<22>;

  ld.param.b64  %rd12, [vecadd_param_1];
  cvta.to.global.u64  %rd1, %rd12;
  ...
  mov.u32   %r1, %ctaid.x;
  mov.u32   %r2, %ntid.x;
  mov.u32   %r4, %tid.x;
  cvt.u64.u32   %rd15, %r4;
  mul.wide.u32  %rd16, %r1, %r2;
  add.s64   %rd20, %rd16, %rd15;
  setp.gt.u64   %p1, %rd20, 1023;
  @%p1 bra  $L__BB0_3;
  ...
$L__BB0_2:
  add.s64   %rd17, %rd1, %rd21;
  ld.global.b32   %r5, [%rd17];
  add.s64   %rd18, %rd2, %rd21;
  ld.global.b32   %r6, [%rd18];
  add.rn.f32  %r7, %r5, %r6;
  add.s64   %rd19, %rd3, %rd21;
  st.global.b32   [%rd19], %r7;
  add.s64   %rd21, %rd21, %rd7;
  add.s64   %rd20, %rd20, %rd5;
  setp.lt.u64   %p2, %rd20, 1024;
  @%p2 bra  $L__BB0_2;
$L__BB0_3:
  ret;
}
```

The `.visible .entry` directive, the `.target sm_80` line, and the 15 descriptor parameters (3 memrefs × 5 slots) confirm the kernel-marker fix and the descriptor ABI.

5.2 Verbatim gfx1101 ISA (AMD, executed locally)

listing 8 is verbatim from `artifacts/run-report-amd/gemm.isa`: the header and the k-accumulate loop with LDS traffic.

Listing 8: gfx1101 ISA for the tiled GEMM (`gemm.isa`).

```
  .amdgcn_target "amdgcn-amd-amdhsa--gfx1101"
  .amdhsa_code_object_version 6
  .text
  .globl  gemm
  .p2align  8
  .type gemm,@function
gemm:
  s_load_b32 s3, s[0:1], 0xb4
  v_and_b32_e32 v7, 31, v0
  v_dual_mov_b32 v1, 0 :: v_dual_and_b32 v6, 0xe0, v0
  s_clause 0x5
  s_load_b64 s[10:11], s[0:1], 0x78
  ...
  v_cvt_f32_u32_e32 v2, s17
  ...
  v_mul_f32_e32 v7, v19, v5
  ...
```

...and the compute loop. The `ds_` loads show the LDS path; `v_fmac_f32` is the accumulator FMA, with 32 occurrences in this ISA:

Listing 9: gfx1101 k-accumulate loop with LDS traffic (`gemm.isa`).

```
  ds_store_b32 v29, v30
  ...
  s_waitcnt lgkmcnt(0)
  s_waitcnt_vscnt null, 0x0
  s_barrier
  buffer_gl0_inv
  ds_load_b128 v[29:32], v12
  ds_load_2addr_b32 v[6:7], v16 offset1:32
  ds_load_2addr_b32 v[37:38], v16 offset0:64 offset1:96
  ds_load_b128 v[33:36], v12 offset:16
  ds_load_2addr_b32 v[39:40], v16 offset0:128 offset1:160
  ...
  v_fmac_f32_e32 v28, v29, v6
  s_delay_alu instid0(VALU_DEP_1) | instskip(SKIP_3) | instid1(VALU_DEP_1)
  v_fmac_f32_e32 v28, v30, v7
  ...
```

ISA-wide totals (`grep -c` on the artifact): `ds_load`/`ds_store`/ `s_barrier` 30 matches, `v_fmac_f32` 32. The embedded amdhsa metadata tail pins the resource usage: `.group_segment_fixed_size: 8192`, `.vgpr_count: 41`, `.sgpr_count: 26`, `.wavefront_size: 32`, `.max_flat_workgroup_size: 256`, and `.kernarg_segment_size: 424`. These values match `resources.json` (vgpr 41, sgpr 26, lds 8192).

5.3 Serialization Mechanics

`gpu-module-to-binary` runs vendor linkers and does not touch a GPU:

- **ROCDL `format=bin`** (HSACO): requires the ROCm toolkit: `<toolkit>/llvm/bin/ld.lld` (an *external* linker, LLD 22.0.0 rocm-7.2.3) plus the device-libs at `<toolkit>/amdgcn/bitcode/` (`ocml.bc`, `ockl.bc`, `hip.bc`, `opencl.bc` + 67 `oclc_*` control bitcodes; 71 files total). The `rocm-toolkit-root` linkFarm aggregate remaps `${rocmPackages.llvm.lld}/bin/ld.lld` → `llvm/bin/ld.lld` and every `${rocmPackages.rocm-device-libs}/amdgcn/bitcode/*.bc` → `amdgcn/bitcode/`. Toolkit is wired via `ROCM_TOOLKIT_ROOT`; a bogus toolkit path fails cleanly with the serializer's own error, and `format=isa` still requires the toolkit because the serializer drives the same ld.lld path.

- **NVVM `format=bin`** (cubin): needs `ptxas` (CUDA 12.9, `Build cuda_12.9.r12.9/compiler.36037853_0`) and libdevice (`nvvm/libdevice/libdevice.10.bc`, passed through the pass's own `l=` linkFiles option, separate from `opts=`), both from the `cuda-toolkit-root` aggregate. `format=isa` (PTX text) needs no toolkit.
- Valid format strings are `offloading|llvm|assembly|isa|binary|bin| fatbinary|fatbin`. The codebase does not use "asm".
- **Blob extraction**: after `gpu-module-to-binary` the module holds `gpu.binary [#gpu.object<target, bin = "...">]`; `compileToBlob` walks for the `BinaryOp`, matches the object's `CompilationTarget`, and reads the raw payload StringAttr (in memory it is raw bytes; MLIR escaping happens at print time). Verified artifacts start with the ELF magic `7f 45 4c 46`; the cubin `llvm-readobj -h` reports `Machine: EM_CUDA (0xBE)`.

Artifact sizes: `gemm.hsaco` 6528 B (ELF, 3 amdhsa note sections), `gemm.isa` 14 877 B, `vecadd.cubin` 3496 B (EM_CUDA), `vecadd.ptx` 1619 B.

6 Runtime and Loading

The runtime is plain C++ (`lib/runtime`, linked with clang and `-lamdhip64`, with HIP type-free headers; it does not use hipcc). The local path:

- `DeviceContext::create` → `hipSetDevice(0)` + live queries (device name "AMD Radeon RX 7800 XT", 30 CUs, 64 KiB LDS, wavefront 32, max threads 1024; all agree with the gfx1101 device-limits table entry).
- `GpuModule::load(image, size)` → **`hipModuleLoadData`** → `get_function` → **`hipModuleGetFunction`** → `KernelLaunch.launch(fn)` → **`hipModuleLaunchKernel`**. The launch uses `sharedMemBytes=0` (the v1 ABI uses static LDS only) and descriptor-packed `kernelParams`.

6.1 Descriptor ABI

Each rank-N buffer uses five slots for 1-D and eleven for rank-2: allocated pointer, aligned pointer, offset i64(0), N sizes, and N strides. Strides are emitted dim-0-first ($16 \times 8 \rightarrow$ strides `[8,1]`). Scalars follow by value. `LaunchArgs::params()` returns the `void**` array pointing at each slot. This matches the MLIR memref descriptor layout used by the vendor lowering (§4).

6.2 CUDA Backend (cross-ready, not linked)

The same interface is implemented over the CUDA Driver API bound at runtime via `dlopen("libcuda.so.1")` + `dlsym` of 24 symbols (`cuInit`, `cuDeviceGet`, `cuModuleLoadData`, `cuModuleGetFunction`, `cuLaunchKernel`, `cuEvent*`, `cuMemAlloc_v2`/`cuMemFree_v2`/ `cuMemcpyHtoD_v2`/`cuMemcpyDtoH_v2`, ...). If `dlopen` fails every call returns a clean error (`kCudaDriverUnavailable`), and `kernelh bench` on NVIDIA targets exits 77 (skip). `ldd` on the binary shows zero `libcuda` references.

Three CUDA backend bugs. The first run on an RTX 6000 Ada pod exposed three bugs that the local dlopen-failure path had hidden:

1. **dlsym names**: the table asked for CamelCase names (`CuInit`, `CuDeviceGet`, ...), but the driver exports lowercase names (`cuInit`, `cuDeviceGet`, ...). The live GPU returned "`dlsym(CuInit) failed`".
2. **`__v2` variants**: `cuMemAlloc`/`cuMemFree`/ `cuMemcpyHtoD`/`cuMemcpyDtoH` (deprecated aliases) return `CUDA_ERROR_INVALID_CONTEXT` (201) even with the primary context current on that driver; `cuMemAlloc_v2`/`cuMemFree_v2`/`cuMemcpyHtoD_v2`/ `cuMemcpyDtoH_v2` succeed. Fixed to the `_v2` variants (verified present via `nm -D`).
3. **Attribute IDs**: the warp size used 39 (= `CU_DEVICE_ATTRIBUTE_MAX_THREADS_PER_MULTIPROCESSOR`, returned 1536) instead of 10 (returned 32); `MultiProcessorCount` used 41 instead of 16 (returned 1; RTX 6000 Ada has 142 SMs). Fixed per `cuda.h` CUDA 12.9.

The full path (DSL, pipeline, emitter, NVVM cubin, descriptor-ABI launch, numeric verification) runs on NVIDIA hardware (§9).

7 Autotuning

`kernelh tune` (`lib/compiler/autotuner.cpp` + the CLI driver in `tools/kernelh/main.cpp`) is GEMM-only in v1.

Config space (`gemm_config_space()`): $216 = 3(\text{bm}) \times 3(\text{bn}) \times 2(\text{bk}) \times 2(\text{waves}) \times 2(\text{unroll}) \times 3(\text{vec})$, with bm/bn $\in \{64,128,256\}$, bk $\in \{16,32\}$, waves $\in \{4,8\}$, unroll $\in \{1,2\}$, vec $\in \{1,2,4\}$. `-max-candidates` caps the number evaluated; `-sample` sizes the fixed-seed random sample; `-refine-budget` bounds best-neighbor refinement.

Evaluation gates, in order, in a forked child with a wall-clock deadline:

1. Compile for the tuning shape; an LDS/shared-memory compile error is surfaced as `pruned: lds`.
2. Launch verification → `pruned: launch`.
3. Resource check (`resources.json` on the actual artifact) → `pruned: vgpr` / `pruned: lds` if the kernel cannot be resident (the profiler-feedback seam).
4. **Mandatory numeric re-verification** on a small shape (≤ 128/dim) against the fp64 reference, tolerance rel-Frobenius $\leq 1e{-}5$. Any candidate that fails is discarded. The gate accepts a candidate only after numeric re-verification.
5. EventTimer timing (warmup 3, median 10) on the tuning shape for accepted candidates.

Cache: whole-run JSON cache (`build/tune-cache.json`, hand-rolled JSON writer) keyed `sha256(kernel+shape+devic`
A second run with the same key is served entirely from cache.

Recorded run. `tune-results.csv` holds the verbatim results for 8 candidates at `m=512,n=512,k=512` with `-max-candidates 8`:

Listing 10: Autotune results: a negative result (`tune-results.csv`).

```
kernel,shape,bm,bn,bk,waves,unroll,vec,ms_median,accepted,reason
gemm,"k=512,m=512,n=512",64,64,16,4,1,2,0,false,reverify: FAIL (rel-frobenius 0.7071607398 > 1e-5)
gemm,"k=512,m=512,n=512",128,128,16,8,1,1,0,false,reverify: FAIL (rel-frobenius 0.8660254038 > 1e-5)
gemm,"k=512,m=512,n=512",128,64,32,8,2,4,0,false,reverify: FAIL (rel-frobenius 0.8837545775 > 1e-5)
gemm,"k=512,m=512,n=512",256,64,32,8,1,2,0,false,reverify: FAIL (rel-frobenius 0.7072175998 > 1e-5)
gemm,"k=512,m=512,n=512",128,256,16,8,2,1,0,false,reverify: FAIL (rel-frobenius 0.9703290074 > 1e-5)
gemm,"k=512,m=512,n=512",256,128,32,8,1,2,0,false,reverify: FAIL (rel-frobenius 0.8660397367 > 1e-5)
gemm,"k=512,m=512,n=512",64,256,32,4,1,4,0,false,reverify: FAIL (rel-frobenius 0.8660456794 > 1e-5)
gemm,"k=512,m=512,n=512",128,64,32,4,2,4,0,false,reverify: FAIL (rel-frobenius 0.8835009408 > 1e-5)
```

All eight candidates failed numeric re-verification. The three unroll=2 candidates (0-based indices 2, 4, and 7; rel-Frobenius 0.883–0.970) trace to the unroll pass: its partial-unroll cleanup loop mis-remaps the k-loop iter_args carry chain and corrupts the accumulator. The five unroll=1 candidates failed the same gate; this report does not diagnose those failures. The gate rejected every candidate, and the search ended with `no accepted candidate` and `skipping refinement`, so no broken configuration shipped. The run logs `cache-miss` on the first pass, `cache-hit` and `tune ok (served from cache)` on the rerun, and `best config: none (no candidate passed all gates)`.

8 Performance

All numbers below are measurements from `build/bench/results.csv`, generated by `kernelh bench -suite all -out build/bench` with warmup 5 and the median of 20 GPU-event-timed launches on an AMD Radeon RX 7800 XT.

8.1 Measured Roofline Ceilings (this box)

Table 2: Measured microbenchmark ceilings (`build/bench/results.csv`).

microbenchmark	shape	median ms	rate
`roofline_triad`	$n = 2^{28}$ per buffer (1 GiB × 3)	9.26	347.8 GB/s
`roofline_fma`	$2^{22} \times 8192$ FMAs/thread	4.42	15 559.8 GFLOPS

The ridge point (arithmetic intensity where the two ceilings meet) is AI = 15 559.8 GFLOPS/347.8 GB/s = 44.7 flops/byte; the `charts_stats.csv` row carries the exact computed value `ceiling_ai = 44.737538` with `ceiling_gflops = 15559.795120`. fig. 3 shows the measured roofline [6], with the measured ceilings and the kernelh GEMM and attention points plotted at their arithmetic intensities.

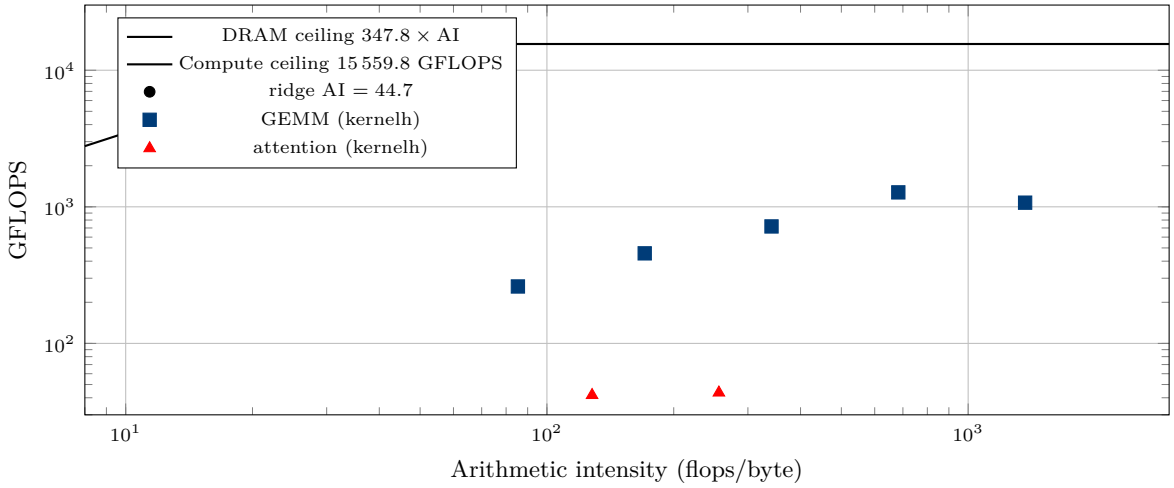


Figure 3: Measured roofline [6] for the RX 7800 XT: DRAM ceiling (347.8 GB/s × AI), FMA compute ceiling (15 559.8 GFLOPS), and the kernelh GEMM/attention points from `charts_stats.csv`. GEMM AI = $N/6$ (square), attention AI = $s/4$.

8.2 GEMM: kernelh vs. rocBLAS

Table 3: GEMM, kernelh vs. rocBLAS (numbers from `build/bench/results.csv`).

shape (m=n=k)	kernelh ms	kernelh GFLOPS	rocBLAS ms	rocBLAS GFLOPS	ratio
512^3	1.028	261.1	0.0576	4658.7	17.8×
1024^3	4.706	456.3	0.281	7646.6	16.8×
2048^3	23.91	718.4	1.925	8925.3	12.4×
4096^3	107.75	1275.5	13.79	9970.2	7.8×
8192^3	1025.16	1072.5	107.83	10 196.5	9.5×

fig. 4 plots the same data (log-scale GFLOPS). The hipref-naive GEMM reference: $512^3 \rightarrow$ 4497.9 GFLOPS, $1024^3 \rightarrow$ 6665.9.

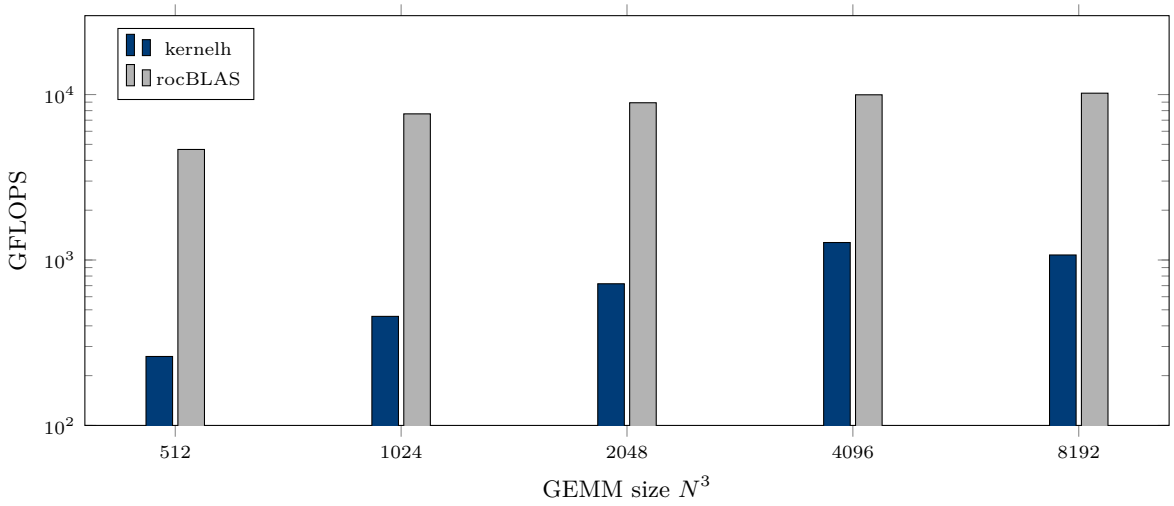


Figure 4: GEMM GFLOPS vs. size: kernelh (scalar, tiled, static shared memory) vs. rocBLAS (same box, same fp32). Data from `build/bench/results.csv`.

The GEMM gap. kernel.h's GEMM is a scalar, tiled, static-shared-memory kernel that uses f32 FMA; tensor cores and WMMA are out of scope (§10). At 8192^3 it is about 9.5× slower than rocBLAS (1072.5 vs. 10 196.5 GFLOPS). The gap is largest at small sizes (17.8× at 512^3), where rocBLAS's tuned inner kernel and launch overhead dominate. The relative gap narrows as N grows: tile efficiency improves, although the 8192^3 kernelh run is the slowest cell in the suite at 1025 ms. The best kernelh cell (1275.5 GFLOPS at 4096^3) reaches about 8% of the measured FMA ceiling (15 559.8 GFLOPS). The causes are scalar accumulation with a $32 \times 32 \times 32$ tile, 12.5% occupancy (§8.4), no double-buffered pipeline, and about 30% of peak LDS bandwidth used by the `ds_load_2addr` pattern. rocBLAS also avoids WMMA on gfx1101; it uses register-blocked accumulation, wider tiles, and aggressive pipelining. Those are v1 scope-outs. The design space is covered by CUTLASS [7], cuBLAS [8], and rocBLAS [9].

8.3 Elementwise / Reduction Kernels

Table 4: Elementwise / reduction kernels (numbers from `build/bench/results.csv`).

kernel	shape	kernelh	hipref-naive	hipref-vec
softmax	4096×1024	8.5 GB/s	475.5 GB/s	717.0 GB/s
softmax	16384×4096	9.6 GB/s	424.2 GB/s	492.4 GB/s
layernorm	4096×1024	69.0 GB/s	472.7 GB/s	709.0 GB/s
layernorm	16384×4096	68.3 GB/s	410.2 GB/s	489.1 GB/s
scan	100 000	3.16 GB/s	3.68 GB/s	3.69 GB/s
scan	1 000 000	3.57 GB/s	4.35 GB/s	4.27 GB/s
fused_gelu	2^{20}	627.9 GB/s	663.7 GB/s	706.9 GB/s
fused_gelu	2^{22}	1110.1 GB/s	1160.8 GB/s	1072.3 GB/s
attention	b=2, h=8, seq=512	41.8 GFLOPS	n/a	n/a
attention	b=2, h=8, seq=1024	43.6 GFLOPS	n/a	n/a

Analysis.

- **fused_gelu** is close to the HIP references and faster at 2^{22}. The fusion pass (§3.1) removes the intermediate tensor's global round-trip. Its 1100 GB/s exceeds the 347.8 GB/s triad ceiling because the triad microbenchmark streams 3 GiB (DRAM-bound) while fused_gelu's 16 MiB working set stays in L1/L2. The triad ceiling is a DRAM ceiling; small working sets can sit above it.
- **softmax and layernorm are the weak spot**: 8.5–69 GB/s against 400–700 GB/s for the HIP references, a 10–80× gap. These kernels are bandwidth-bound. The HIP references use float4 vectorization; the kernelh templates run scalar (`vector_width=1` in the default schedule). The measured rates show a v1 codegen inefficiency.

- **scan** is near parity (3.16–3.57 vs. 3.68–4.35 GB/s). The two-pass Blelloch cost shows up as the flat low rate; both implementations are bandwidth-bound.
- **attention** has no local vendor baseline. rocBLAS lacks an attention API, and the cuBLAS/CUTLASS rows skip with exit 77 because this box has no NVIDIA driver. The online, tiled, fp32 attention reaches about 43.6 GFLOPS at seq=1024 (AI = $s/4 = 256$ flops/byte, in the compute region of the roofline). The achieved point sits below the FMA ceiling, as expected for a small, low-occupancy kernel at 3.125% occupancy (§8.4). It is a FlashAttention-lite [2, 3] realization with online rescaling, per-tile LDS staging, and fp32.

8.4 Occupancy

From `build/bench/occ/occupancy.md`. Theoretical occupancy comes from the resource reports and the device-limits table. Measured occupancy is "n/a" on this box: `hipOccupancyMaxActiveBlocksPerMultiprocessor` returns `hipErrorInvalidDeviceFunction` for every kernel under ROCm 7.2.3 and this kernel. The analytic fallback is the only working path.

Table 5: Occupancy table (theoretical; measured n/a on this driver stack).

kernel	config	vgpr	lds (B)	theoretical %	measured %	limiter
gemm	$512^3 \ldots 8192^3$	41	8192	12.5	n/a	vgpr
softmax	4096×1024 / 16384×4096	48	0	12.5	n/a	vgpr
layernorm	4096×1024 / 16384×4096	12	0	12.5	n/a	threads
scan	10^5 / 10^6	21	3072	12.5	n/a	threads
fused_gelu	2^{20} / 2^{22}	10	0	12.5	n/a	threads
attention	seq=512/1024	27	33 024	3.125	n/a	lds

Analysis. 12.5% theoretical occupancy is low. The GEMM limiter is VGPRs (41 VGPRs/kernel, so $\lfloor 6144/(41 \times 32) \rfloor = 4$ waves of 32 per CU against 32 waves/CU). The attention kernel is LDS-limited at 3.125%: its K/V/Q staging footprint is 33 024 B of the 65 536 B budget, which admits one block per CU. The v1 static-LDS design sets these ceilings. Register blocking and shared-memory double buffering are future work (§10). The measured-to-theoretical gap comes from the platform.

9 Validation and Failure Evidence

This box runs in full runtime mode. `.omo/evidence/degraded.flag` contains `degraded=false` (15 bytes, written by the AMD runtime smoke gate). All `[gpu]`-tagged e2e tests execute on the GPU; none skip.

- **AMD runtime gate**: `rocminfo` shows Agent gfx1101 (AMD Radeon RX 7800 XT) with `KERNEL_DISPATCH`; `HSA_OVERRIDE_GFX_VERSION` is unset; the hipcc vectorAdd binary prints `VECTORADD-PASS` 2/2 (2^{20} elements, host-side exact compare); a HIP module-loader probe compiles.
- **MLIR ROCDL pipeline gate**: a hand-authored `gpu.module` with a cross-thread LDS exchange lowered to gfx1101 HSACO (10 208 B, ELF magic, 3 amdhsa note sections, `group_segment_fixed_size: 1024`, 8 `ds_` instructions, `s_barrier`), loaded via the production module loader and executed: `LOADMODULE-PASS` 2/2 with full-array host-side numeric verification. A bogus toolkit path fails cleanly naming the ROCm path.
- **Kernel e2e on the live GPU** (Catch2 `[gpu]`): vecadd `max-abs-diff=0.000000000` at $1024/65\,536/2^{20}$; softmax worst `max-abs-diff=0.000000004`; GEMM `rel-frobenius=0.000000e+00` at all 6 sizes including the 1000^3 boundary; attention `max-abs-diff` $\sim 1.5e{-}7$; the `[!shouldfail]` negative controls all fire; the canary prints `CANARY-GPU-PASSED`. Full suite: 324/324 ctest tests pass (unit + lit + on-GPU e2e).
- **Runtime e2e**: the MLIR-emitted `xchg.hsaco` runs through the descriptor-ABI runtime (`RUNTIMEXCHG-PASS`, $N = 1 << 20$).
- **NVIDIA live validation** (RTX 6000 Ada pod, PrimeIntellect): the full path from DSL through pipeline, emitter, and NVVM cubin to the descriptor-ABI launch and numeric verification runs on

NVIDIA hardware: `NV-BACKEND-OK`, `NV-DEVICE` (cu=142 lds=49 152 wavefront=32 maxThreads=1024), `NV-MODULE-LOAD-OK`, `NV-LAUNCH-OK` (grid=4096 block=256), `NV-VECADD-PASS` (N=2^{20}, host-side verified). Driver 580.126.09 (CUDA 13.0).

- **Bench and runtime failures exit with a message**: clock pinning emits EPERM warnings and falls back to median-of-20; a missing CUDA driver exits 77 with a message; a hung autotune candidate is SIGKILLed and recorded `timeout`; broken per-pass dumps make `kernelh explore` exit 1 and name the file.

One failure did occur. The original serial GEMM caused a GPU hang that killed the desktop session at 8192^3: about 44 s per launch, 25 launches, which exceeded the AMD lockup_timeout. Root cause: the nested global loads could not be staged by promote-shared, so the HSACO had `group_segment_fixed_size: 0` and no `ds_` instructions, and all 256 threads redundantly computed the whole product at about 12 GFLOPS. The fix rewrites the template to pre-stage tiles through static LDS. The fixed kernel runs 8192^3 in 1005–1025 ms with `group_segment_fixed_size: 8192` and 18–30 `ds_` instructions.

10 Limitations

1. **Tensor cores are out of scope.** gfx1101 WMMA is untrusted upstream (ROCm/ROCm#6349 [10]); the plan greps kernels/lib for `wmma`/`mfma` and requires zero hits. GEMM and attention use scalar f32 FMAs, which caps them below the vendor libraries (§8.2).

2. **NVIDIA code is cross-compiled here.** No NVIDIA hardware is local. Every NVIDIA artifact is emitted and the blobs are verified (ELF/PTX magic, `.target sm_80`, `Machine: EM_CUDA`), then exercised on a rented RTX 6000 Ada pod, where the full path runs and numerically verifies (`NV-VECADD-PASS`). `kernelh bench` NVIDIA rows exit 77 locally because no CUDA driver is present.

3. **Unofficial distro and kernel.** The host is a rolling distro with an in-tree AMD driver and kernel. It is smoke-gated and works (§9), but it is not an officially supported driver stack, and the HIP occupancy API returns `hipErrorInvalidDeviceFunction`. The analytic fallback is the working path.

4. **Static shared memory only in v1.** The launch uses `sharedMemBytes=0` and the module uses addrspace-3 globals. Dynamic shared memory is not used; this avoids the hidden kernel-argument ABI and fixes the LDS budget at compile time (attention uses 33 024 of 65 536 B).

5. **No double buffering in the v1 GEMM.** One k-tile is staged, followed by a barrier, compute, and another barrier. The memory pipeline is idle during compute. This is the largest remaining GEMM performance lever.

6. **No `hipMallocAsync`.** ROCm/ROCm#6178 is an in-tree driver regression; the runtime uses plain `hipMalloc` and has no stream-ordered allocation.

7. **Occupancy API unavailable.** Measured occupancy is analytic-only (§8.4).

8. **rocprofv3 is not packaged here.** The resource reporter falls back to `llvm-readobj -notes` on the HSACO and `ptxas -v` on the NVIDIA side (§5.3).

11 Future Work

Prioritized by measured impact (§8 evidence):

1. **Fix the unroll-pass iter_args corruption.** The autotuner's numeric gate currently rejects every unroll=2 GEMM config (§7). Fixing the main and cleanup-loop remap tables unlocks the unroll dimension of the autotune space.

2. **Close the GEMM gap** (9.5×, §8.2). Add double-buffered k-tile pipelining, register-blocked accumulation to cut the 41-VGPR footprint and lift occupancy off 12.5%, and a wider tile (BM/BN $>$ 32) with bank-conflict-free layouts. Target: approach rocBLAS's ∼10.2 TFLOPS without WMMA.

3. **Vectorize the bandwidth-bound kernels** (softmax/layernorm 10–80× gap, §8.3). The vectorize pass exists, but the default schedules use `vector_width=1`. Enabling float4 loads and stores on the elementwise kernels is the codegen change with the highest measured impact.
4. **Improve attention** (43.6 GFLOPS, 3.125% occupancy). Shrink the 33 024 B K/V/Q staging footprint with smaller per-block tiles or register-resident segments, so more than one block fits per CU. Add a cross-vendor baseline by running the cuBLAS/CUTLASS harness on NVIDIA hardware (FlashAttention-2-style [3] tiling).
5. **WMMA and MFMA.** Deferred until ROCm/ROCm#6349 [10] is resolved upstream. Re-audit gfx1101 correctness before enabling tensor cores.
6. **Widen the autotuner.** Add non-uniform sampling; the current unroll=2 corner is dead. Persist per-config timing for accepted configs; the CSV already has the schema. Add softmax and attention to the tunable set.
7. **Measure occupancy on a supported stack.** The local ROCm and rolling-kernel combination returns `hipErrorInvalidDeviceFunction`. Replace the analytic column with API numbers.
8. **Run NVIDIA locally.** When NVIDIA hardware is available, run the cuda-cross rows instead of exit-77 skips.

12 Conclusion

`kernel.h` takes one kernel description from a C++23 DSL to PTX and AMDGCN machine code for two vendors, runs it on the GPU, checks it against an fp64 reference, and benchmarks it against a vendor library. The front half (DSL, atom IR, seven passes) has no MLIR dependency. MLIR handles the atom dialect, the `gpu`/`scf`/`vector` device lowering, and the vendor serializers. The autotuner's numeric gate caught an unroll-pass bug, and the per-pass dump self-check catches corruption. Bench and runtime failures exit with a message. The performance gaps (softmax/layernorm vectorization, GEMM occupancy, double buffering) are measured in Section 8, and Section 11 lists the next steps.

Appendix: Artifact Map

table 6 maps every artifact referenced in this report to its producing command.

Table 6: Artifact map.

artifact	produced by	contents
`artifacts/run-report-amd/` (18 files)	`kernelh explore -kernel gemm -shape m=256,n=256,k=256 -target amdgcn-gfx1101`	per-pass .atom dumps, MLIR stages, gemm.isa, gemm.hsaco, resources.json, occupancy, index.md
`artifacts/run-report-nv/` (18 files)	`kernelh explore -kernel vecadd -shape n=1024 -target nvptx-sm_80`	same set, vecadd.ptx, vecadd.cubin, resources.json, occupancy (6.25% threads)
`artifacts/run-report-fused/`	`kernelh explore -kernel fused_gelu -shape n=1048576 -target amdgcn-gfx1101`	fusion before and after dumps (02 to 03)
`artifacts/tune-report/tune-results.csv`	`kernelh tune -kernel gemm -shape m=512,n=512,k=512 -max-candidates 8`	8 evaluated configs, all rejected by numeric re-verification (§7)
`build/bench/results.csv` (40 rows)	`kernelh bench -suite all -out build/bench`	roofline, kernelh, rocBLAS, and hipref rows (§8)
`build/bench/charts/*.png`, `charts_stats.csv`	`python3 scripts/charts.py build/bench/results.csv build/bench/charts`	measured-ceiling roofline and per-backend charts
`build/bench/occ/occupancy.json`, `occupancy.md`	`kernelh bench -suite kernelh -occupancy -out build/bench/occ`	15-row occupancy table (§8.4)

References

[1] Chris Lattner et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation”. In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. arXiv:2002.11054. 2021, pp. 2–14. DOI: 10.1109/CGO51591.2021.9370308. arXiv: 2002.11054.

[2] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. arXiv:2205.14135. 2022. arXiv: 2205.14135.

[3] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *International Conference on Learning Representations (ICLR)*. arXiv:2307.08691. 2024. arXiv: 2307.08691.

[4] Riyadh Baghdadi et al. “Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code”. In: *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. arXiv:1804.10694. 2019, pp. 193–205. DOI: 10.1109/CGO.2019.8661197. arXiv: 1804.10694.

[5] Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86. DOI: 10.1109/CGO.2004.1281665.

[6] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009). CACM 52(4), not arXiv; cited by DOI, pp. 65–76. DOI: 10.1145/1498765.1498785.

[7] NVIDIA. *CUTLASS: CUDA Templates for Linear Algebra Subroutines*. https://github.com/NVIDIA/cutlass. Open-source library; v3+ features the CuTe layout/tiling backend. 2025.

[8] NVIDIA. *cuBLAS: CUDA Basic Linear Algebra Subroutine Library*. https://docs.nvidia.com/cuda/cublas/. Vendor GEMM baseline used by the cross-vendor benchmark harness. 2025.

[9] AMD. *rocBLAS: ROCm Basic Linear Algebra Subroutine Library*. https://github.com/ROCm/rocBLAS. Local vendor GEMM baseline on the RX 7800 XT. 2025.

[10] AMD. *ROCm: Open Software Platform for GPU-Centric Computing*. https://github.com/ROCm/ROCm. Includes the ROCm/ROCm#6349 gfx1101 WMMA correctness tracking issue. 2025.